

QA DOCUMENTACIÓN

Sprint 1

Responsabilidades:

Diseño y ejecución de casos de prueba

Hubo casos de prueba funcionales para cada módulo del sistema, cubriendo tanto el flujo esperado, como escenarios de error y bordes. Los módulos evaluados incluyeron:

- Gestión de perfiles: registro de nuevos usuarios, inicio de sesión, cambio de contraseña temporal, validación de reglas de contraseña (longitud, ausencia de vocales, mínimo de letras/números, sin caracteres repetidos consecutivos), y bloqueo de acceso para perfiles inactivos.
- Gestión de ubicación: selección en cascada de provincia → cantón → distrito, geolocalización en el mapa, persistencia de ubicaciones nulas para perfiles recién creados, y ocultamiento de la ubicación cuando es desactivada administrativamente.
- Módulo de música: carga del carrusel de canciones, reproducción mediante la API de YouTube, acumulación de tiempo de reproducción y conteo de reproducciones (incluyendo el caso de repetición al finalizar el video).
- Panel administrativo: CRUD de perfiles, géneros y canciones, con verificación de que las operaciones de baja no eliminan físicamente los registros, sino que alternan su estado (activo/inactivo).
- Control de accesos por roles: restricción de vistas exclusivas de administrador y de cliente según el rol de la sesión activa.

Cada caso de prueba documentó: precondiciones, pasos de ejecución, resultado esperado, resultado obtenido y estado (aprobado/fallido).

Verificación del cumplimiento de los requerimientos funcionales

Se validó que cada módulo cumpliera con los requerimientos definidos por el equipo de análisis, contrastando el comportamiento del sistema contra las historias de usuario establecidas. Ejemplos de verificaciones realizadas:

- Un perfil con `tbperfilactivo = 0` no puede autenticarse, mostrando el mensaje "Su cuenta está inactiva. Contacte al administrador."
- Las canciones cuyo género está desactivado no se muestran al cliente final, aunque la canción en sí esté activa.
- El registro de tiempo de reproducción se acumula en incrementos de 10 segundos por perfil y canción, sin duplicar registros gracias a la restricción `UNIQUE(tbperfilid, tbcancionid)`.

- Los combos de ubicación no muestran provincias, cantones o distritos marcados como inactivos.

Identificación, documentación y seguimiento de errores

Durante el ciclo de pruebas se identificaron y corrigieron incidencias como:

- Inconsistencia entre el número de parámetros del constructor de la entidad Perfil y las llamadas realizadas desde el controlador, que provocaba el registro incorrecto del rol de usuario.
- Ausencia de manejo de valores NULL en las consultas de ubicación mediante INNER JOIN, que ocultaba registros de perfiles sin ubicación asignada.
- Bloques de código duplicados dentro de una misma estructura switch, donde PHP ejecutaba únicamente la primera coincidencia y dejaba código muerto en la segunda, ocasionando que ciertas actualizaciones (como el estado de ubicación) no se reflejaran en la base de datos.
- Endpoints de tipo API que devolvían una redirección HTTP (código 302) en lugar de una respuesta JSON al validar el rol del usuario, lo que rompía el análisis de la respuesta en el cliente; se solucionó diferenciando la validación de rol para vistas HTML de la validación para endpoints de datos.

Cada incidencia fue reproducida, analizada de su causa raíz y verificada nuevamente tras su corrección antes de cerrarse.

Validación de la integración de los diferentes módulos del sistema

Se comprobó que los módulos interactúan correctamente entre sí, por ejemplo:

- Que la creación de un perfil también genera su registro correspondiente de ubicación (con valores nulos) y su registro inicial de accesos semanales.
- Que la desactivación de un perfil no rompe la integridad referencial con las tablas de reproducciones, accesos y ubicación asociadas.
- Que el panel de administración de perfiles refleja correctamente el estado de la ubicación de cada usuario mediante la consulta conjunta entre tbperfil y tububicacion.
- Que el conteo de reproducciones y el tiempo acumulado se actualizan de forma consistente al alternar entre pausar, reanudar y repetir una canción.

Verificación del cumplimiento de los estándares de calidad establecidos para el proyecto

Se verificó que el código siguiera el patrón arquitectónico establecido para el proyecto, y que la interfaz de administración mantuviera consistencia visual entre los distintos módulos CRUD (perfiles, géneros, canciones).

Sprint 2

Responsabilidades

Durante el segundo sprint se incorporaron módulos de perfilado conductual, gestión de eventos y registro semanal en tiempo real. Los módulos evaluados incluyeron:

- Perfilado musical, gastronómico y deportivo mediante redes neuronales: cada ámbito cuenta con su propio modelo entrenado con los datos del perfil. Gastronomía y deportes utilizan una mecánica tipo Tinder donde el usuario desliza para indicar preferencias, y la red neuronal determina su afinidad. Los resultados se presentan como características del usuario en el panel de administración.
- Sistema generalizado de perfilado: un único controlador y API despachan al modelo correspondiente según el tipo solicitado, permitiendo agregar nuevos ámbitos sin modificar la vista ni el API.
- Módulo de eventos: el administrador crea eventos con múltiples ubicaciones físicas, cada una asociada a un tipo de contenido. Cuando un cliente ingresa a una ubicación, el sistema valida automáticamente su entrada y genera una afinidad al contenido correspondiente.
- Gestión de feriados: se incorporó una lista de feriados que modifica el tratamiento de las franjas horarias en los registros, dado que los patrones de consumo en feriados difieren de los días regulares.
- Registros semanales en tiempo real: se amplió el sistema de registros para cubrir los tres ámbitos principales (reproducciones, gastronomía y deportes), con tablas independientes por ámbito.

Verificación del cumplimiento de los requerimientos funcionales

- El modelo de perfilado requiere un mínimo de eventos registrados para generar resultados; con menos datos muestra un mensaje informativo al administrador.
- La mecánica de deslizamiento registra correctamente la preferencia positiva o negativa y la asocia al perfil activo.
- El ingreso a una ubicación de evento genera automáticamente la afinidad correspondiente sin intervención manual.
- En días feriados el sistema aplica franjas horarias diferenciadas de forma transversal a los tres módulos de registro.
- El selector de tipo de perfilado mantiene el ID del perfil al navegar entre tipos y resalta correctamente el tipo activo.

Identificación, documentación y seguimiento de errores

- Al renombrar el módulo de conciertos a eventos, quedaron referencias desactualizadas en vistas, controladores y APIs que apuntaban a los nombres anteriores. Se rastrearon y actualizaron todas las referencias de forma sistemática para mantener la consistencia del sistema.
- Los imports y dependencias al inicio de los controladores y APIs requerían los nombres exactos de los archivos y clases. Cambios de nomenclatura en el proyecto provocaron errores de carga que se resolvieron revisando y actualizando cada declaración `require_once` afectada.

- Los resultados generados por las redes neuronales variaban entre ejecuciones debido a la inicialización aleatoria de los pesos del modelo. Se ajustaron los parámetros de entrenamiento y se fijaron semillas de aleatoriedad para estabilizar los resultados entre sesiones.
- La conexión con el gestor de base de datos presentó problemas iniciales de configuración, ya que el sistema requería credenciales y parámetros de host específicos del entorno. Se investigó la configuración correcta y se actualizó el archivo de conexión hasta lograr una conexión estable.
- Algunas tablas de la base de datos tenían restricciones de integridad referencial que impedían insertar o eliminar registros en ciertos flujos del sistema. Se revisaron las restricciones de claves foráneas y se ajustaron las operaciones para respetar el orden correcto de inserción y eliminación.
- Columnas definidas como NOT NULL sin valor por defecto causaban errores al insertar registros desde flujos donde ciertos campos aún no estaban disponibles. Se revisaron los esquemas de las tablas afectadas y se definieron valores por defecto apropiados.
- Durante las pruebas de integración se detectaron inconsistencias entre los nombres de columnas utilizados en las consultas SQL y los nombres reales en la base de datos, producto de renombramientos durante el diseño. Se corrigieron las consultas afectadas y se estandarizó la nomenclatura.
- Algunas credenciales de acceso a la base de datos diferían entre el entorno de desarrollo local y el entorno compartido del equipo, causando fallos de conexión intermitentes. Se centralizó la configuración de credenciales en el archivo de configuración del proyecto.

Validación de la integración de los diferentes módulos del sistema

- El controlador generalizado carga dinámicamente el modelo correcto sin acoplamiento entre módulos.
- Los registros de la mecánica de deslizamiento alimentan correctamente las redes neuronales de cada ámbito.
- El ingreso a ubicaciones de eventos queda vinculado al perfil y se refleja en los datos de perfilado.
- Los registros semanales de los tres ámbitos se actualizan de forma independiente sin interferir entre sí.
- El tratamiento de feriados se aplica de forma consistente en los tres módulos de registro.

Verificación del cumplimiento de los estándares de calidad

- Separación de responsabilidades: cada modelo encapsula su lógica de red neuronal; el controlador despacha y la vista solo presenta.
- Extensibilidad: agregar un nuevo tipo de perfilado requiere únicamente crear el modelo y registrar una entrada en el controlador.
- Seguridad: todos los endpoints validan el rol y devuelven JSON con éxito: false sin redireccionamientos.
- Registro continuo: toda interacción queda registrada semanalmente en tiempo real para mantener los modelos actualizados.